JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Predicate<T>

Type Parameters:

T - the type of the input to the predicate

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface Predicate<T>
```

Represents a predicate (boolean-valued function) of one argument.

This is a functional interface whose functional method is `test(Object)`.

Since:

1.8

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
`default Predicate<T>`	**`and`**`(Predicate<? super T> other)`	Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another.
`static <T> Predicate<T>`	**`isEqual`**`(Object targetRef)`	Returns a predicate that tests if two arguments are equal according to `Objects.equals(Object, Object)`.
`default Predicate<T>`	**`negate`**`()`	Returns a predicate that represents the logical negation of this predicate.
`static <T> Predicate<T>`	**`not`**`(Predicate<? super T> target)`	Returns a predicate that is the negation of the supplied predicate.

default Predicate<T>	or(Predicate<? super T> other)	Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another.
boolean	test(T t)	Evaluates this predicate on the given argument.

Method Details

test

`boolean test(T t)`

Evaluates this predicate on the given argument.

Parameters:

`t` - the input argument

Returns:

`true` if the input argument matches the predicate, otherwise `false`

and

`default Predicate<T> and(Predicate<? super T> other)`

Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. When evaluating the composed predicate, if this predicate is `false`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

Parameters:

`other` - a predicate that will be logically-ANDed with this predicate

Returns:

a composed predicate that represents the short-circuiting logical AND of this predicate and the `other` predicate

Throws:

`NullPointerException` - if other is null

negate

```
default Predicate<T> negate()
```

Returns a predicate that represents the logical negation of this predicate.

Returns:

a predicate that represents the logical negation of this predicate

or

```
default Predicate<T> or(Predicate<? super T> other)
```

Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. When evaluating the composed predicate, if this predicate is `true`, then the `other` predicate is not evaluated.

Any exceptions thrown during evaluation of either predicate are relayed to the caller; if evaluation of this predicate throws an exception, the `other` predicate will not be evaluated.

Parameters:

`other` - a predicate that will be logically-ORed with this predicate

Returns:

a composed predicate that represents the short-circuiting logical OR of this predicate and the `other` predicate

Throws:

`NullPointerException` - if other is null

isEqual

```
static <T> Predicate<T> isEqual(Object targetRef)
```

Returns a predicate that tests if two arguments are equal according to `Objects.equals(Object, Object)`.

Type Parameters:

T - the type of arguments to the predicate

Parameters:

`targetRef` - the object reference with which to compare for equality, which may be `null`

Returns:

a predicate that tests if two arguments are equal according to `Objects.equals(Object, Object)`

not

`static <T> Predicate<T> not(Predicate<? super T> target)`

Returns a predicate that is the negation of the supplied predicate. This is accomplished by returning result of the calling `target.negate()`.

Type Parameters:

T - the type of arguments to the specified predicate

Parameters:

`target` - predicate to negate

Returns:

a predicate that negates the results of the supplied predicate

Throws:

`NullPointerException` - if target is null

Since:

11

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.